**Group Rembers:**

202402040028_Saishwari Korade

202402040027_Shruti Biradar

# Multi-Task Vision System using YOLOv8

## Introduction

Computer Vision is a rapidly growing field of Artificial Intelligence that enables machines to interpret and understand visual data such as images and videos. One of the most powerful and widely used techniques in this domain is object detection using the YOLO (You Only Look Once) algorithm. YOLO models are known for their high speed and accuracy, making them suitable for real-time applications.

In this project, a multi-task vision system is developed using modern YOLO models (v8 and above). The system is capable of performing multiple computer vision tasks including object detection, image classification, pose estimation, and oriented bounding box (OBB) detection. The implementation is done using the Ultralytics YOLO framework on a local machine without using any cloud platforms.

This project demonstrates how a single framework can be effectively used to solve different vision problems and highlights the versatility of YOLO models in real-world applications.

## Objective

The main objective of this project is to design and implement a multi-task computer vision system using YOLO models (v8 or newer versions) that can perform the following tasks:

- Object Detection
- Image Classification
- Pose Estimation
- Oriented Bounding Box (OBB) Detection

Additional objectives include:

- To train lightweight YOLO models (nano/small) on custom datasets
- To evaluate model performance using test data
- To deploy the trained model locally using webcam for real-time detection

- To understand the working of modern deep learning-based vision systems

## Datasets

The datasets used in this project are obtained from Roboflow, which provides pre-processed and annotated datasets suitable for YOLO training.

Four different datasets are used for the respective tasks:

- **Object Detection Dataset**
  Contains images with labeled bounding boxes for detecting objects.
- **Classification Dataset**
  Organized into class-based folders, where each folder represents a category.
- **Pose Estimation Dataset**
  Includes images with keypoint annotations used to detect human body joints.
- **OBB Dataset (Oriented Bounding Boxes)**
  Contains rotated bounding box annotations for detecting objects with orientation.

All datasets are downloaded in YOLO-compatible format and split into training, validation, and testing sets.

## Methodology

The project follows a structured approach for implementing the multi-task vision system:

1. Data Preparation
   - Datasets are downloaded from Roboflow in YOLO format
   - Data is organized into train, validation, and test folders
   - Configuration files (data.yaml) are used for training
2. Model Selection
   - Lightweight YOLO models (YOLOv11 nano versions) are selected
   - Different model variants are used for detection, classification, pose, and OBB
3. Model Training
   - Each task is trained separately using the Ultralytics YOLO library
   - Training is performed on a local system (CPU)
   - Hyperparameters such as epochs and image size are configured

4. Model Evaluation
    - Models are tested on unseen data
    - Performance is evaluated using prediction outputs and accuracy metrics
5. Prediction and Output Generation
    - The trained models generate predictions on test images
    - Outputs include bounding boxes, class labels, and keypoints
6. Deployment
    - The detection model is deployed locally using webcam
    - Real-time object detection is demonstrated as proof of deployment

## Dataset Links

Task 1: Object Detection

**https://app.roboflow.com/saishwari-mukesh-korade/cars-in-traffic-xtatd-ocvof/1**

Task 2: Image Classification

**https://app.roboflow.com/saishwari-mukesh-korade/vehicle-classification-eapcd-ntq32/1**

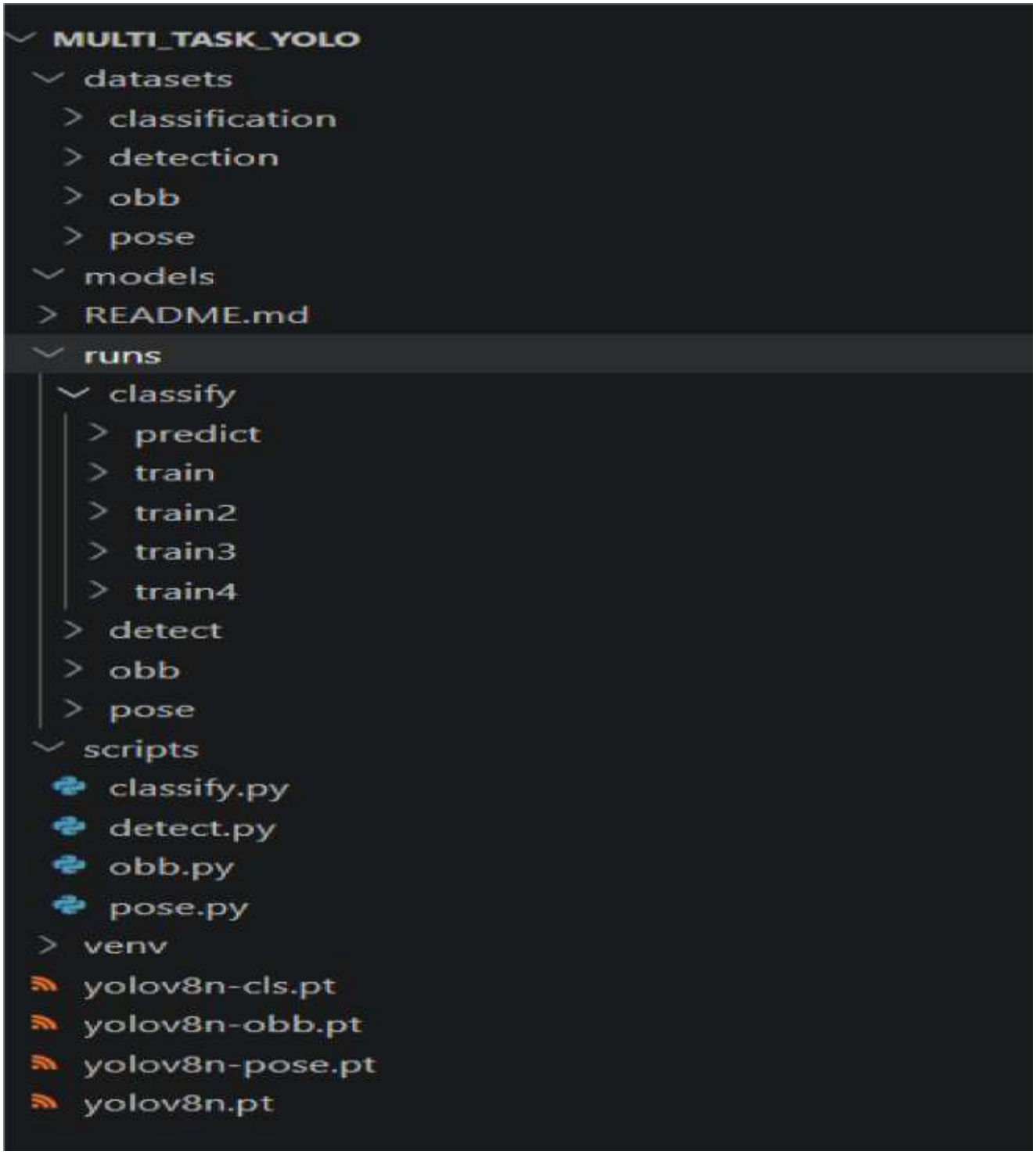
Task 3: Pose Estimation

**https://app.roboflow.com/saishwari-mukesh-korade/human-pose-i11rt-53kyi/models**

Task 4: Oriented Bounding Boxes (OBB)

**https://universe.roboflow.com/cocotoyolov8/oriented-bounding-box-gdrpy**

## Folder Structure



# Source Code

## Task 1: Object Detection

### detect.py

```
from ultralytics import YOLO

# Load YOLO model  model =
YOLO("yolov8n.pt")

# Train model.train(
data=r"C:\Users\saib9\Desktop\multi_task_yolo\datasets\detection\data.yaml", epochs=5,
imgsz=640 )

# Predict on test images model.predict(
source=r"C:\Users\saib9\Desktop\multi_task_yolo\datasets\detection\test\ima ges",
save=True )

# Webcam deployment  model =
YOLO(r"C:\Users\saib9\Desktop\multi_task_yolo\runs\detect\train\weights\best
.pt") model.predict(source=0)
```

## Task 2: Image Classification

### classify.py

```
from ultralytics import YOLO

#  LOAD BASE MODEL
model = YOLO("yolov8n-cls.pt")

# TRAIN MODEL
model.train(
data=r"C:\Users\saib9\Desktop\multi_task_yolo\datasets\classification",     epochs=3,
imgsz=224 )

#  LOAD TRAINED MODEL (best.pt)
```

```
trained_model = YOLO(
r"C:\Users\saib9\Desktop\multi_task_yolo\runs\classify\train2\weights\best.p t" )

# STEP 4: PREDICT  trained_model.predict(
source=r"C:\Users\saib9\Desktop\multi_task_yolo\datasets\classification\test
\cars_test",     save=True)

print("Classification Task Completed Successfully!")
```

## Task 3: Pose Estimation

**Pose.py**

```
from ultralytics import YOLO

# LOAD POSE MODEL
model = YOLO("yolov8n-pose.pt")

# TRAIN MODEL
model.train(
   data=r"C:\Users\saib9\Desktop\multi_task_yolo\datasets\pose\data.yaml",
epochs=3,     imgsz=640 )

# PREDICT  model.predict(
source=r"C:\Users\saib9\Desktop\multi_task_yolo\datasets\pose\test\images", save=True )

# WEBCAM
model = YOLO(r"C:\Users\saib9\Desktop\multi_task_yolo\runs\pose\train\weights\best. pt")
model.predict(source=0, show=True)
print("Pose Estimation Completed!")
```

## Task 4: Oriented Bounding Boxes (OBB)

**Obb.py** from ultralytics import YOLO

```
# LOAD OBB MODEL
model = YOLO("yolov8n-obb.pt")

# TRAIN MODEL
model.train( data=r"C:\Users\saib9\Desktop\multi_task_yolo\datasets\obb\data.yaml",
epochs=3,     imgsz=640 )
```

```
# PREDICT
model.predict(
    source=r"C:\Users\saib9\Desktop\multi_task_yolo\datasets\obb\test\images",
save=True)
print("OBB TASK COMPLETED!")
```

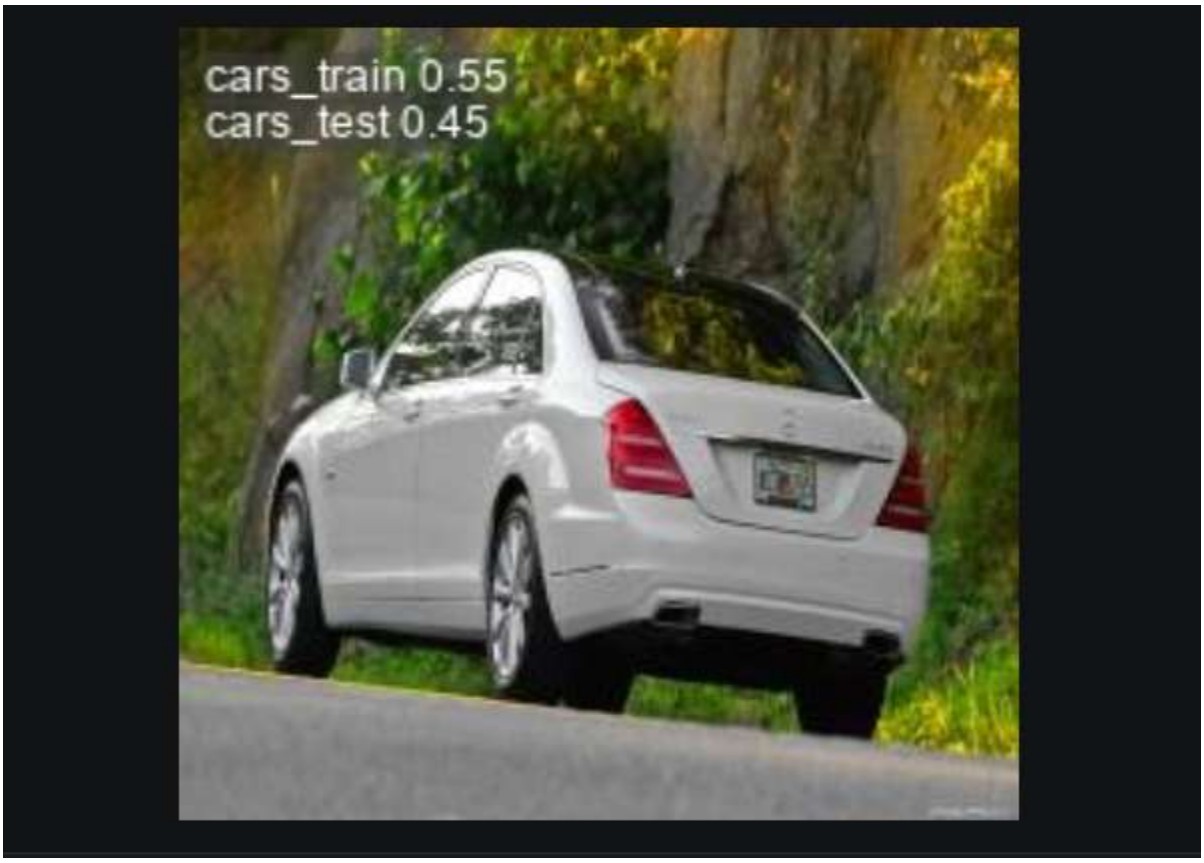
## Project Implementation Video Link

[https://drive.google.com/file/d/1qx1md225hrwzlzlkJbce1kkOQLY-lo8V/view?usp=sharing](https://drive.google.com/file/d/1qx1md225hrwzlzlkJbce1kkOQLY-lo8V/view?usp=sharing)

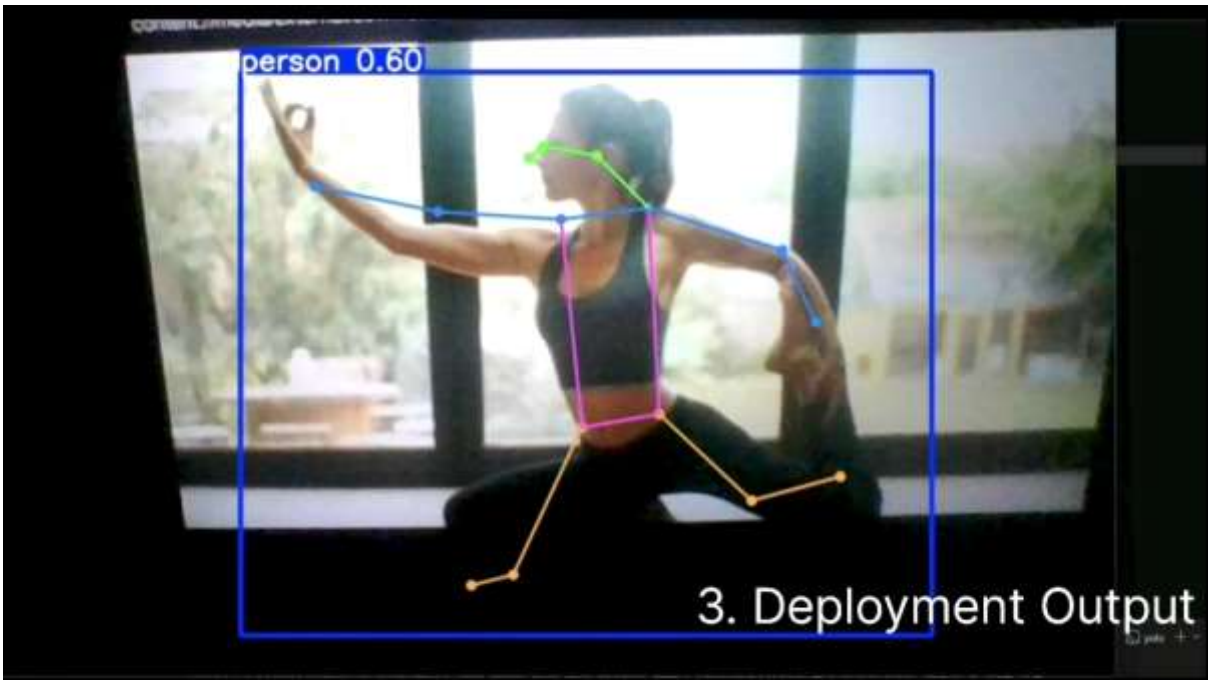
## Deployment Proof

**Task 1  Output**

**Task 2 Output**




**Task 3 Output**



**Task 4 Output**